Traffic Analyzer — Documentação do Desafio

Oct 21, 2026 · @Rafael de Souza Rodrigues

Visão geral

O Traffic Analyzer captura pacotes de uma interface de rede, grava cada pacote em SQLite e exibe estatísticas de tráfego no terminal. Foi escrito em Python 3.12 com Scapy 2.6.1 e roda em Docker.

A tabela abaixo liga cada requisito do desafio ao ponto do código que o atende.

Requisito	Como foi atendido	Onde
Capturar pacotes de uma interface especificada	Scapy `AsyncSniffer`, interface escolhida por `-i`	`app/main.py`
Usar biblioteca adequada	Scapy, com filtros BPF opcionais via libpcap	`app/main.py`
Capturar IP origem, IP destino, protocolo e tamanho	Função `parse_packet`, com suporte a IPv4, IPv6 e ARP	`app/parser.py`
Total de pacotes capturados	`COUNT(*)` por sessão	`app/stats.py`
Pacotes por protocolo	`GROUP BY protocol`, com percentual e bytes	`app/stats.py`
Top 5 IPs de origem e de destino	`GROUP BY src_ip` e `dst_ip`, ordenado por bytes ou pacotes	`app/stats.py`
Armazenar os pacotes em banco	SQLite, uma linha por pacote, gravação em lotes	`app/database.py`, `app/writer.py`
Executar em Docker	`Dockerfile` + `docker-compose.yml` com rede do host	raiz do projeto
Documentação	Este documento e o `README.md`	raiz do projeto

Funcionalidade

A aplicação captura pacotes de rede e exibe as estatísticas pedidas. Isso foi validado em uma captura real de 8 segundos, que registrou 2.031 pacotes sem nenhum descarte.

O fluxo de uso tem quatro comandos:

- `interfaces` lista as interfaces disponíveis.
- `capture` captura, grava no banco, mostra um painel ao vivo a cada 5 segundos e imprime o relatório final.
- `stats` recalcula o relatório a partir do banco, para uma sessão ou para todas.
- `sessions` lista as capturas já gravadas.

A captura termina pelo limite de pacotes (`-c`), pelo tempo (`-t`), por Ctrl+C ou por `docker stop` . Em todos os casos, ela grava o último lote, fecha a sessão e imprime o relatório.

Exemplo de saída real

```
================================================================
 RESULTADO - sessão #2 (eth0)
================================================================
  Total de pacotes capturados: 2031
  Volume total:                6.4 MB

  Pacotes por protocolo
  Protocolo  Pacotes      %    Bytes
  ---------  -------  -----  -------
  TCP           1911  94.1%   6.4 MB
  UDP            120   5.9%  10.4 KB

  Top 5 IPs de ORIGEM (ordenado por bytes)
  #  Endereço IP     Pacotes     Bytes  % bytes
  -  --------------  -------  --------  -------
  1  140.82.113.3        651    5.8 MB    89.7%
  2  151.101.64.223      317  563.7 KB     8.6%
  3  192.0.2.2          1003  111.7 KB     1.7%
  4  8.8.8.8              60    6.4 KB     0.1%

  Top 5 IPs de DESTINO (ordenado por bytes)
  #  Endereço IP     Pacotes    Bytes  % bytes
  -  --------------  -------  -------  -------
  1  192.0.2.2          1028   6.3 MB    98.3%
```

```
  2  140.82.113.3         549  58.4 KB     0.9%
  3  151.101.64.223       394  49.3 KB     0.7%
  4  8.8.8.8               60    4.0 KB     0.1%
============================================================
```

O ranking mostra 4 IPs porque essa captura teve apenas 4 endereços distintos. Com mais tráfego, ele mostra 5.

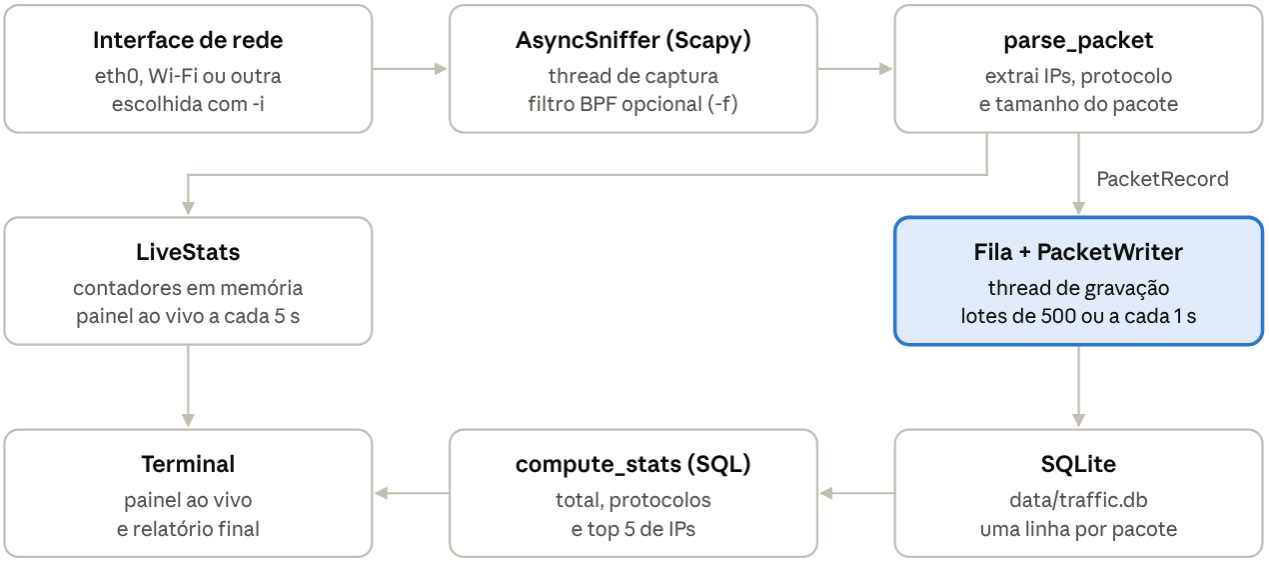
Verificações realizadas

Cenário	Resultado
Captura na interface `eth0` com tráfego HTTPS e DNS	2.031 pacotes gravados, 0 descartados
Filtro BPF `-f udp` na interface `lo`	Apenas UDP capturado; TCP descartado pelo filtro
Encerramento por SIGTERM (equivalente ao `docker stop`)	Sessão fechada, relatório impresso, código de saída 0
Interface inexistente	Erro claro antes de iniciar, código de saída 2
Suíte de testes automatizados	9 de 9 testes aprovados

Arquitetura

Três threads dividem o trabalho: a de captura lê os pacotes, a de gravação salva no banco e a principal mostra o painel e o relatório.

A captura nunca espera o disco: a gravação roda em outra thread



arquitetura · 3 threads, 7 componentes

Cada pacote vira um `PacketRecord`, que segue por dois caminhos: os contadores em memória do painel ao vivo e a fila que alimenta o banco. O relatório final sai do banco, via SQL.

Estrutura de arquivos

Arquivo	Responsabilidade
`app/main.py`	CLI, orquestração da captura e tratamento de sinais
`app/parser.py`	Converte o pacote do Scapy em `PacketRecord`
`app/writer.py`	Thread que grava os pacotes em lotes
`app/database.py`	Schema e acesso ao SQLite
`app/stats.py`	Estatísticas via SQL e contadores ao vivo
`app/report.py`	Formatação das tabelas no terminal
`tests/`	Testes com pacotes sintéticos
`Dockerfile`, `docker-compose.yml`	Imagem e execução com rede do host
`data/`	Banco SQLite, montado como volume
`README.md`	Documentação resumida dentro do repositório

Configuração

A forma recomendada de executar é com Docker em um host Linux, porque assim o container enxerga as interfaces reais da máquina. No Windows também funciona, com as diferenças descritas abaixo.

Pré-requisitos

Ambiente	O que instalar	Observação
Linux	Docker 20.10+ e Docker Compose v2	Captura o tráfego real da máquina
Windows com Docker	WSL2 e Docker Desktop (backend WSL2)	Captura o tráfego da VM do Docker, não o da placa física
Windows nativo (opcional)	Python 3.12 e Npcap	Captura o tráfego real do Wi-Fi ou da Ethernet
Qualquer sistema, sem Docker	Python 3.12 e libpcap (Linux)	Precisa de root ou de permissão de captura

Linux

1. Instale o Docker e o plugin Compose pelo gerenciador de pacotes da distribuição.
2. Extraia o projeto e entre na pasta `traffic-analyzer`.
3. Rode `docker compose build`.

Windows com Docker Desktop

1. Confirme que a virtualização está habilitada. Ela aparece no Gerenciador de Tarefas, em Desempenho e depois em CPU.
2. No PowerShell como Administrador, rode `wsl --install` e reinicie o computador.
3. Instale o Docker Desktop com a opção "Use WSL 2" marcada e espere o ícone ficar verde.
4. Extraia o projeto em um caminho sem espaços, por exemplo `C:\projetos\traffic-analyzer`.
5. Nessa pasta, rode `docker compose build`.

No Windows, o Docker roda dentro de uma VM Linux. Por isso, o modo `network_mode: host` aponta para a rede dessa VM. O tráfego do navegador não aparece na captura; para testar, gere tráfego a partir de outro container na mesma rede (veja Execução e uso).

Windows nativo (para ver o tráfego real)

1. Instale o Python 3.12 marcando "Add python.exe to PATH".
2. Instale o Npcap (npcap.com) com as opções padrão.
3. Na pasta do projeto, crie o ambiente virtual e instale as dependências:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements-dev.txt
```

Se o PowerShell bloquear o `Activate.ps1`, rode uma vez `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

Variáveis de ambiente

Variável	Padrão	Uso
`DB_PATH`	`/data/traffic.db` no container	Caminho do banco SQLite
`INTERFACE`	interface padrão do sistema	Interface usada quando `-i` não é informado

Execução e uso

Todos os comandos seguem o formato `docker compose run --rm analyzer <comando> [opções]`. Sem Docker, troque o prefixo por `python -m app.main`.

Roteiro de execução

```
docker compose build
docker compose run --rm --entrypoint pytest analyzer -v   # 9 testes devem
passar
docker compose run --rm analyzer interfaces               # descobre o nome
da interface
docker compose run --rm analyzer capture -i eth0 -t 60    # captura por 60
segundos
docker compose run --rm analyzer stats                    # relatório da
última captura
docker compose run --rm analyzer sessions                 # lista as
capturas gravadas
```

Para gerar tráfego durante a captura, use outro terminal. No Linux, qualquer comando serve, como `curl https://example.com`. No Windows com Docker Desktop, o tráfego precisa vir de um container na mesma rede:

```
docker run --rm --network host alpine sh -c "ping -c 20 8.8.8.8; wget -qO- https://example.com > /dev/null"
```

Outros exemplos

```
# 1000 pacotes, somente TCP e UDP
docker compose run --rm analyzer capture -i eth0 -c 1000 -f "tcp or udp"

# até Ctrl+C, painel ao vivo a cada 10 segundos
docker compose run --rm analyzer capture -i eth0 --live-interval 10

# relatório de todas as sessões, top 10 por número de pacotes
docker compose run --rm analyzer stats --all --order-by packets --top 10

# Windows nativo, interface Wi-Fi
python -m app.main capture -i "Wi-Fi" -t 60
```

Referência de opções

Comando	Opção	Descrição
global	`--db CAMINHO`	Banco SQLite a usar
global	`-v`	Logs detalhados
`capture`	`-i, --interface`	Nome, descrição ou nome NPF da interface
`capture`	`-c, --count N`	Para após N pacotes; 0 significa sem limite
`capture`	`-t, --timeout S`	Para após S segundos
`capture`	`-f, --filter BPF`	Filtro BPF, como `tcp`, `port 443` ou `host 10.0.0.5`
`capture`	`--live-interval S`	Intervalo do painel ao vivo; 0 desativa; padrão 5
`capture`	`--batch-size N`	Pacotes por transação no banco; padrão 500
`capture`, `stats`	`--top N`	Tamanho do ranking de IPs; padrão 5

Comando	Opção	Descrição
`capture`, `stats`	`--order-by`	`bytes` (padrão) ou `packets`
`stats`	`-s ID`	Sessão específica; padrão é a mais recente
`stats`	`--all`	Agrega todas as sessões

Consultando os dados gravados

O banco fica em `data/traffic.db`, na pasta do projeto, porque essa pasta é montada como volume. Ele pode ser aberto no DB Browser for SQLite, na extensão SQLite Viewer do VS Code ou no `sqlite3`. Mantenha os arquivos `traffic.db-wal` e `traffic.db-shm` na mesma pasta.

```
-- Últimos 50 pacotes, com horário legível
SELECT datetime(captured_at, 'unixepoch', 'localtime') AS horario,
       src_ip, dst_ip, protocol, size
FROM packets ORDER BY id DESC LIMIT 50;

-- Conversas com mais bytes em uma sessão
SELECT src_ip, dst_ip, protocol, COUNT(*) AS pacotes, SUM(size) AS bytes
FROM packets WHERE session_id = 1
GROUP BY src_ip, dst_ip, protocol ORDER BY bytes DESC LIMIT 10;

-- Tráfego por minuto
SELECT strftime('%H:%M', captured_at, 'unixepoch', 'localtime') AS minuto,
       COUNT(*) AS pacotes, SUM(size) AS bytes
FROM packets WHERE session_id = 1 GROUP BY minuto;
```

Banco de dados

O banco é SQLite, com duas tabelas: `capture_sessions` guarda cada execução da captura e `packets` guarda uma linha por pacote. Esse desenho atende o requisito de armazenar os pacotes e permite recalcular qualquer estatística depois, sem capturar de novo.

Schema

```
CREATE TABLE capture_sessions (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    interface     TEXT    NOT NULL,
    bpf_filter    TEXT,
```

```
    started_at    TEXT    NOT NULL,          -- ISO-8601 UTC
    ended_at      TEXT,                      -- NULL enquanto a captura roda
    packet_count  INTEGER NOT NULL DEFAULT 0,
    dropped_count INTEGER NOT NULL DEFAULT 0
);

CREATE TABLE packets (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    session_id      INTEGER NOT NULL REFERENCES capture_sessions(id) ON
DELETE CASCADE,
    captured_at     REAL    NOT NULL,        -- epoch em segundos (precisão
de µs)
    src_ip          TEXT,
    dst_ip          TEXT,
    protocol        TEXT    NOT NULL,
    protocol_number INTEGER,
    ip_version      INTEGER,
    size            INTEGER NOT NULL CHECK (size >= 0)
);

CREATE INDEX idx_packets_session  ON packets(session_id);
CREATE INDEX idx_packets_protocol ON packets(session_id, protocol);
CREATE INDEX idx_packets_src      ON packets(session_id, src_ip);
CREATE INDEX idx_packets_dst      ON packets(session_id, dst_ip);
```

Colunas da tabela `packets`

Coluna	Tipo	Exigida no desafio	Por que existe
`src_ip`	TEXT	Sim	IP de origem; NULL em quadros que não são IP
`dst_ip`	TEXT	Sim	IP de destino; NULL em quadros que não são IP
`protocol`	TEXT	Sim	Nome legível: TCP, UDP, ICMP, ARP etc.
`size`	INTEGER	Sim	Tamanho do quadro em bytes, incluindo o cabeçalho Ethernet
`session_id`	INTEGER	Não	Separa capturas diferentes
`captured_at`	REAL	Não	Horário do kernel; permite análises por tempo

Coluna	Tipo	Exigida no desafio	Por que existe
`protocol_number`	INTEGER	Não	Número IANA; preserva o dado exato quando não há nome conhecido
`ip_version`	INTEGER	Não	Separa IPv4 de IPv6 sem interpretar o texto do endereço

Justificativas

- **Uma linha por pacote, não contadores.** O desafio pede o armazenamento dos pacotes. Guardar o dado bruto permite novas análises, como tráfego por minuto ou por conversa.
- **Tabela de sessões.** Registra o contexto de cada captura (interface, filtro, início, fim e descartes) e permite relatórios por sessão ou agregados. O `ON DELETE CASCADE` apaga uma sessão inteira com um único comando.
- **IPs como texto.** Funciona igual para IPv4 e IPv6 e fica legível em consultas manuais. O SQLite não tem tipo nativo para endereços.
- **Colunas que aceitam NULL.** Quadros não IP, como LLDP, também são gravados, para que o total de pacotes seja o total real. Os rankings de IP ignoram os NULL.
- **Índices compostos com `session_id` primeiro.** Todas as consultas filtram por sessão e agrupam por protocolo ou IP. Esses índices atendem exatamente esse padrão.

Por que SQLite

O SQLite não precisa de servidor, cabe em um único arquivo montado como volume e já vem com o Python. Assim, o projeto tem uma única dependência externa, o Scapy. O modo `journal_mode=WAL` permite rodar `stats` enquanto outra captura escreve.

Para produção, com vários sensores ou milhões de pacotes por hora, a evolução natural seria PostgreSQL ou um banco de séries temporais, como TimescaleDB ou ClickHouse. Como todo o acesso ao banco está em `app/database.py`, essa troca não afeta o resto do código.

Decisões técnicas

A decisão central foi separar a captura da gravação em threads diferentes, para que a escrita em disco nunca atrase a leitura de pacotes. As demais decisões estão abaixo, cada uma com o motivo.

Decisão	Motivo	Alternativa descartada
Scapy com `AsyncSniffer`	Biblioteca indicada no desafio; decodifica as camadas e permite montar pacotes falsos para testes. O modo assíncrono deixa a thread principal livre para o painel e os sinais	`sniff()` bloqueante, que impede o painel ao vivo e o encerramento limpo
`store=False` na captura	O armazenamento é o banco; guardar os pacotes também em memória esgotaria a RAM em capturas longas	Lista de pacotes em memória
Callback leve + fila + gravação em lotes	Um INSERT com commit por pacote faria o disco limitar a captura e o kernel descartaria pacotes em rajadas. O lote grava até 500 pacotes ou a cada 1 segundo	INSERT por pacote
Fila limitada a 100 mil itens	Sob carga extrema, a aplicação descarta de forma contabilizada (`dropped_count`) em vez de esgotar a memória	Fila sem limite
Duas fontes de estatística	O painel ao vivo usa contadores em memória, sem consultar o banco; o relatório final usa SQL, que é a fonte da verdade. Um teste garante que as duas dão o mesmo resultado	Consultar o banco a cada atualização do painel
"Mais tráfego" medido em bytes	Bytes refletem o consumo de banda. As tabelas mostram também os pacotes, e `--order-by packets` muda o critério	Ranking só por pacotes
`network_mode: host` no Docker	Na rede bridge padrão, o container só enxerga a própria interface virtual	Rede bridge

Decisão	Motivo	Alternativa descartada
Capabilities `NET_RAW` e `NET_ADMIN`	Concedem apenas o necessário para captura e modo promíscuo	`privileged: true`, que dá acesso total ao host
libpcap instalada na imagem	O Scapy precisa dela para compilar filtros BPF; sem ela, a opção `-f` falha	Imagem sem libpcap

Como o protocolo é identificado

- **IPv4:** pelo campo `proto` do cabeçalho IP.
- **IPv6:** pelo campo `next header`, percorrendo os cabeçalhos de extensão (Hop-by-Hop, Routing, Fragment e Destination Options) até chegar à camada de transporte.
- **Nomes:** números conhecidos viram nomes (TCP, UDP, ICMP, ICMPv6, GRE e outros). Os demais aparecem como `IP-<número>`.
- **ARP:** registrado com os IPs que carrega (`psrc` e `pdst`).
- **Outros quadros:** registrados como `ETHERTYPE-0x....`, sem IP.

Robustez do script

- A interface é validada antes da captura, aceitando nome, descrição ou nome NPF. Isso deixa a mesma CLI funcionando no Linux (`eth0`) e no Windows (`Wi-Fi`).
- Um pacote malformado é ignorado sem derrubar a captura.
- Erros da thread de captura, como falta de permissão, chegam à thread principal e são exibidos com uma dica de correção.
- SIGINT (Ctrl+C) e SIGTERM (`docker stop`) apenas sinalizam a parada. A aplicação grava o que restou na fila, fecha a sessão e imprime o relatório. O compose dá 15 segundos para isso (`stop_grace_period`).

Testes, limitações e solução de problemas

Os 9 testes automatizados rodam sem rede e sem root, porque usam pacotes montados com o Scapy e SQLite em memória. Para executá-los: `docker compose run --rm --entrypoint pytest analyzer -v`.

Arquivo	O que verifica
`tests/test_parser.py`	Extração de campos para IPv4/TCP, UDP, ICMP, IPv6 com cabeçalho de extensão, protocolo desconhecido, ARP e quadros não IP
`tests/test_stats.py`	Totais, contagem por protocolo, ranking por bytes e por pacotes, limite do top N, isolamento entre sessões e igualdade entre estatísticas ao vivo e via SQL

Limitações conhecidas

- **Docker Desktop (Windows e macOS):** a rede do host é a da VM do Docker. Para analisar a placa física, use um host Linux ou a execução nativa com Npcap.
- **Interface `lo`:** o Linux entrega cada pacote de loopback duas vezes (saída e entrada). A contagem dobra, assim como no tcpdump.
- **Volume de tráfego:** o Scapy é escrito em Python e atende bem até dezenas de milhares de pacotes por segundo. Para enlaces de vários Gbps, o indicado seria captura em C ou eBPF.
- **Crescimento do banco:** cada pacote ocupa cerca de 100 bytes. Em capturas longas, use `-t`, `-c` ou `-f` e apague sessões antigas.

Solução de problemas

Sintoma	Causa	Solução
`Operation not permitted`	Faltam permissões de captura	Use o `docker-compose.yml` fornecido ou `--cap-add NET_RAW --cap-add NET_ADMIN`
`libpcap is not available. Cannot compile filter`	Execução fora do Docker sem libpcap	Instale `libpcap0.8` ou rode sem `-f`
Interface não encontrada	Nome diferente do esperado	Rode `interfaces` e use o valor da coluna Nome
Captura com 0 pacotes no Windows	Tráfego gerado fora da VM do Docker	Gere tráfego com um container usando `--network host`
Erro de conexão com o daemon do Docker	Docker Desktop fechado	Abra o Docker Desktop e espere o ícone ficar verde